In [51]:
```python
import pandas as pd
# LOAD MASTER DATA (2020–2025 FULL DATASET)
file_path = r"C:\Users\hp\OneDrive\Desktop\DA Project 1 Master Dataset(Copy 1)
df = pd.read_excel(file_path)
print("Data Loaded Successfully")
print(df.head())
print(df.info())
```

```
Data Loaded Successfully
  Symbol Series       Date  Prev Close  Open Price  High Price  Low Price  \
0    TCS     EQ 2020-01-01     2161.70     2168.00     2183.90     2154.0
1    TCS     EQ 2020-01-02     2167.60     2179.95     2179.95     2149.2
2    TCS     EQ 2020-01-03     2157.65     2164.00     2223.00     2164.0
3    TCS     EQ 2020-01-06     2200.65     2205.00     2225.95     2187.9
4    TCS     EQ 2020-01-07     2200.45     2200.50     2214.65     2183.8

   Last Price  Close Price  Average Price  Total Traded Quantity  \
0     2170.00      2167.60        2170.54                1354908
1     2157.00      2157.65        2158.63                2380752
2     2201.00      2200.65        2199.26                4655761
3     2201.35      2200.45        2204.89                3023209
4     2205.00      2205.85        2203.53                2429317

     Turnover ₹  No. of Trades Deliverable Qty % Dly Qt to Traded Qty
0  2.940882e+09          44438          164490                  12.14
1  5.139156e+09          99242         1204079                  50.58
2  1.023921e+10         123516         1833823                  39.39
3  6.665855e+09         135360         1000021                  33.08
4  5.353064e+09          95018          966753                   39.8
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6915 entries, 0 to 6914
Data columns (total 15 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Symbol                  6915 non-null   object
 1   Series                  6915 non-null   object
 2   Date                    6915 non-null   datetime64[ns]
 3   Prev Close              6915 non-null   float64
 4   Open Price              6915 non-null   float64
 5   High Price              6915 non-null   float64
 6   Low Price               6915 non-null   float64
 7   Last Price              6915 non-null   float64
 8   Close Price             6915 non-null   float64
 9   Average Price           6915 non-null   float64
 10  Total Traded Quantity   6915 non-null   int64
 11  Turnover ₹              6915 non-null   float64
 12  No. of Trades           6915 non-null   int64
 13  Deliverable Qty         6915 non-null   object
 14  % Dly Qt to Traded Qty  6915 non-null   object
dtypes: datetime64[ns](1), float64(8), int64(2), object(4)
memory usage: 810.5+ KB
None
```

In [52]:
```python
# ==============================
```

```python
# STEP-2 : FEATURE ENGINEERING
# ==============================
# Daily Return %
df["Daily_Return"] = (df["Close Price"] - df["Prev Close"]) / df["Prev Close"]
# Price Change
df["Price_Change"] = df["Close Price"] - df["Open Price"]
# Moving Averages
df["MA_3"] = df.groupby("Symbol")["Close Price"].transform(lambda x: x.rolling
df["MA_7"] = df.groupby("Symbol")["Close Price"].transform(lambda x: x.rolling
# Lag Feature(Previous day close)
df["Prev_Close_Lag1"] = df.groupby("Symbol")["Close Price"].shift(1)
# Target Column (Next Day Close Price)
df["Target"] = df.groupby("Symbol")["Close Price"].shift(-1)
# Remove NaN rows created by rolling & shifting
df = df.dropna()
# Quick Check
print(df.head())
print(df.columns)
print(df.shape)
```

```
   Symbol Series       Date  Prev Close  Open Price  High Price  Low Price  \
6     TCS     EQ 2020-01-09     2255.25     2248.75     2251.95     2210.0
7     TCS     EQ 2020-01-10     2214.35     2228.00     2234.00     2208.0
8     TCS     EQ 2020-01-13     2213.55     2217.85     2218.95     2184.7
9     TCS     EQ 2020-01-14     2190.35     2195.00     2229.80     2195.0
10    TCS     EQ 2020-01-15     2206.90     2213.00     2231.00     2194.2

    Last Price  Close Price  Average Price  ...    Turnover ₹  No. of Trades  \
6      2220.00      2214.35        2223.77  ...  8.303924e+09         134309
7      2213.00      2213.55        2219.51  ...  4.252150e+09         102938
8      2193.00      2190.35        2197.65  ...  6.249873e+09         113645
9      2210.00      2206.90        2212.56  ...  6.523622e+09         132106
10     2225.05      2226.90        2212.65  ...  5.798645e+09         114594

    Deliverable Qty % Dly Qt to Traded Qty Daily_Return  Price_Change  \
6           1805834                  48.36    -0.018135        -34.40
7            767417                  40.06    -0.000361        -14.45
8           1486327                  52.26    -0.010481        -27.50
9           1276382                  43.29     0.007556         11.90
10          1513859                  57.77     0.009062         13.90

           MA_3         MA_7  Prev_Close_Lag1   Target
6   2225.150000  2200.257143          2255.25  2213.55
7   2227.716667  2206.821429          2214.35  2190.35
8   2206.083333  2211.492857          2213.55  2206.90
9   2203.600000  2212.385714          2190.35  2226.90
10  2208.050000  2216.164286          2206.90  2238.80

[5 rows x 21 columns]
Index(['Symbol', 'Series', 'Date', 'Prev Close', 'Open Price', 'High Price',
       'Low Price', 'Last Price', 'Close Price', 'Average Price',
       'Total Traded Quantity', 'Turnover ₹', 'No. of Trades',
       'Deliverable Qty', '% Dly Qt to Traded Qty', 'Daily_Return',
       'Price_Change', 'MA_3', 'MA_7', 'Prev_Close_Lag1', 'Target'],
      dtype='object')
(6887, 21)
```

In [3]:
```
train_df = df.copy()
```

In [4]:
```
# STEP-4 : FEATURE SELECTION
# ==========================
# Features used for training
feature_cols = [
    "Prev Close",
    "Open Price",
    "High Price",
    "Low Price",
    "Last Price",
    "Close Price",
    "Average Price",
    "Total Traded Quantity",
    "Turnover ₹",
    "No. of Trades",
```

```
    "Deliverable Qty",
    "% Dly Qt to Traded Qty",
    "Daily_Return",
    "Price_Change",
    "MA_3",
    "MA_7",
    "Prev_Close_Lag1"
]
# Target column
target_col = "Target"
# Create X and y
X = train_df[feature_cols]
y = train_df[target_col]
print("X Shape:", X.shape)
print("y Shape:", y.shape)
print("\nFeatures Used:")
print(feature_cols)
```

```
X Shape: (6887, 17)
y Shape: (6887,)

Features Used:
['Prev Close', 'Open Price', 'High Price', 'Low Price', 'Last Price', 'Close Pr
ice', 'Average Price', 'Total Traded Quantity', 'Turnover ₹', 'No. of Trades',
'Deliverable Qty', '% Dly Qt to Traded Qty', 'Daily_Return', 'Price_Change', 'M
A_3', 'MA_7', 'Prev_Close_Lag1']
```

In [5]:
```
print(X.isnull().sum().sum())
print(y.isnull().sum())
```

```
0
0
```

In [6]:
```
# STEP-5 : TRAIN VALIDATION SPLIT (TIME SERIES SAFE)
from sklearn.model_selection import train_test_split
# DO NOT shuffle (important for time series)
X_train, X_val, y_train, y_val = train_test_split(
    X,
    y,
    test_size=0.2,
    shuffle=False
)
print("X_train:", X_train.shape)
print("X_val:", X_val.shape)
print("y_train:", y_train.shape)
print("y_val:", y_val.shape)
```

```
X_train: (5509, 17)
X_val: (1378, 17)
y_train: (5509,)
y_val: (1378,)
```

In [7]:
```
# FIX DATA TYPES BEFORE TRAINING
# Keep only numeric columns
X_train = X_train.select_dtypes(include=["number"])
```

```
X_val   = X_val.select_dtypes(include=["number"])
print("Numeric Features Used:")
print(X_train.columns)
```

```
Numeric Features Used:
Index(['Prev Close', 'Open Price', 'High Price', 'Low Price', 'Last Price',
       'Close Price', 'Average Price', 'Total Traded Quantity', 'Turnover ₹',
       'No. of Trades', 'Daily_Return', 'Price_Change', 'MA_3', 'MA_7',
       'Prev_Close_Lag1'],
      dtype='object')
```

In [53]:

```
# STEP-6 : TRAIN FIRST ML MODEL
from sklearn.ensemble import RandomForestRegressor
# Initialize Model
model = RandomForestRegressor(
    n_estimators=200,
    max_depth=12,
    random_state=42,
    n_jobs=-1
)
# Train Model
model.fit(X_train, y_train)
print("✅ Model Training Completed!")
```

```
✅ Model Training Completed!
```

In [9]:

```
# STEP-7 : MODEL PREDICTION
# Predict on validation data
y_pred = model.predict(X_val)
print("Prediction completed!")
print("First 10 Predictions:")
print(y_pred[:10])
```

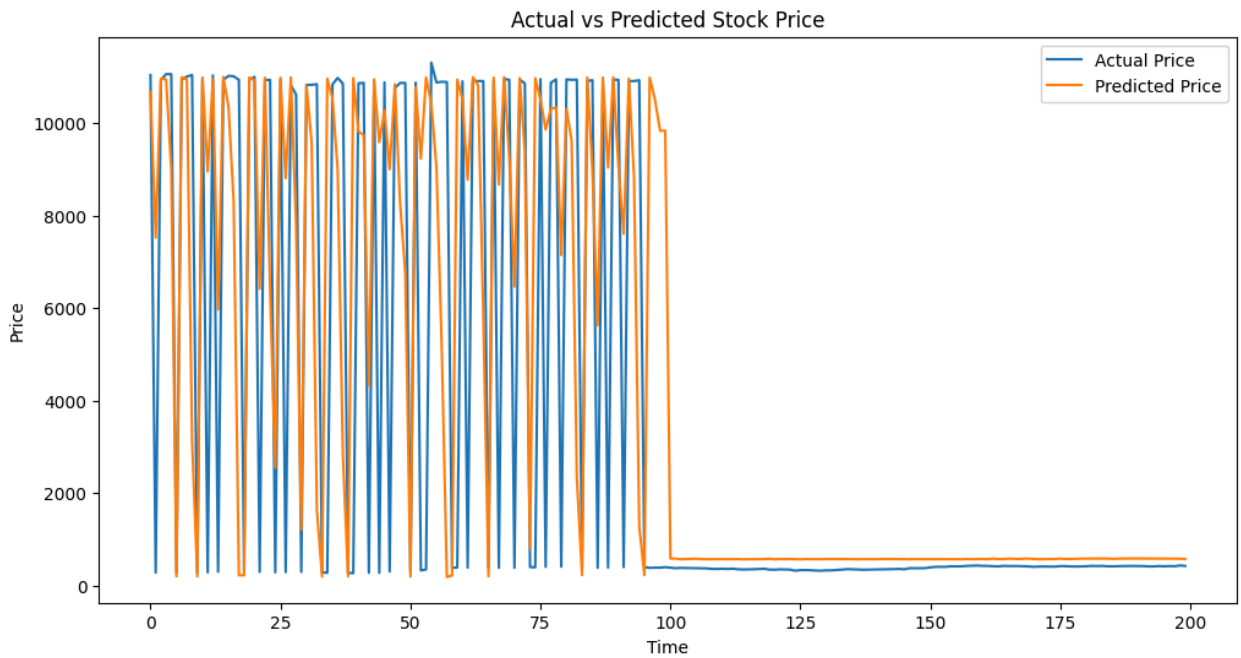
```
Prediction completed!
First 10 Predictions:
[10672.05754335  7520.51227468 10954.37872647 10935.35893005
  8993.87160427   209.13968403 10983.53507185 10938.03067192
  3132.8438705    206.41336845]
```

In [10]:

```
# MODEL PERFORMANCE METRICS
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
mae = mean_absolute_error(y_val, y_pred)
rmse = np.sqrt(mean_squared_error(y_val, y_pred))
r2 = r2_score(y_val, y_pred)
print("Model Performance:")
print("MAE :", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

```
Model Performance:
MAE : 300.6522656986216
RMSE: 1405.0695317134168
R2 Score: 0.56085719955495
```

In [11]:
```
import matplotlib.pyplot as plt
plt.figure(figsize=(12,6))
plt.plot(y_val.values[:200], label="Actual Price")
plt.plot(y_pred[:200], label="Predicted Price")
plt.title("Actual vs Predicted Stock Price")
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
plt.show()
```



In [12]:
```
import pandas as pd
```

In [13]:
```
# STEP-8 : LOAD 2026 SENTIMENT DATA
file_2026 = r"C:\Users\hp\OneDrive\Desktop\Master dataset 2(Sentiment analysis
df_2026 = pd.read_excel(file_2026)
df_2026.columns = df_2026.columns.str.strip()
df_2026["Date"] = pd.to_datetime(df_2026["Date"])
print("2026 Data Loaded Successfully!")
print(df_2026.head())
print(df_2026.shape)
```

```
2026 Data Loaded Successfully!
  Symbol Series       Date  Prev Close  Open Price  High Price  Low Price  \
0    TCS     EQ 2026-01-01      3206.2      3215.0      3234.1     3198.2
1    TCS     EQ 2026-01-02      3227.4      3234.0      3254.3     3219.5
2    TCS     EQ 2026-01-03      3250.7      3240.5      3247.0     3191.0
3    TCS     EQ 2026-01-04      3216.1      3210.0      3263.2     3205.0
4    TCS     EQ 2026-01-05      3255.8      3242.9      3300.6     3223.2

   Last Price  Close Price  Average Price  ...    Turnover ₹  No. of Trades  \
0      3225.0       3227.4        3219.97  ...  4.043425e+09          56642
1      3245.7       3250.7        3241.10  ...  3.838517e+09          63356
2      3213.0       3216.1        3220.21  ...  7.540784e+09         115281
3      3256.8       3255.8        3242.40  ...  9.442929e+09         163408
4      3294.0       3295.6        3284.64  ...  6.534592e+09          87338

   Deliverable Qty  % Dly Qt to Traded Qty  News_Count  Positive_Count  \
0           497590                   39.63           0               0
1           743388                   62.77           0               0
2          1488966                   63.58           0               0
3          2176224                   74.72           0               0
4          1187319                   59.68           0               0

   Negative_Count  Neutral_Count  Sentiment_Score  Sentiment_Label
0               0              0              0.0          Neutral
1               0              0              0.0          Neutral
2               0              0              0.0          Neutral
3               0              0              0.0          Neutral
4               0              0              0.0          Neutral

[5 rows x 21 columns]
(140, 21)
```

In [14]:

```
# STEP-9 : FEATURE ENGINEERING FOR 2026
# Daily Return %
df_2026["Daily_Return"] = (df_2026["Close Price"] - df_2026["Prev Close"]) / d
# Price Change
df_2026["Price_Change"] = df_2026["Close Price"] - df_2026["Open Price"]
# Moving averages (group by Symbol)
df_2026["MA_3"] = df_2026.groupby("Symbol")["Close Price"].transform(lambda x:
df_2026["MA_7"] = df_2026.groupby("Symbol")["Close Price"].transform(lambda x:
# Previous close lag
df_2026["Prev_Close_Lag1"] = df_2026.groupby("Symbol")["Close Price"].shift(1)
# Drop NaNs created
df_2026 = df_2026.dropna()
print("2026 Features Created!")
print(df_2026.head())
```

```
2026 Features Created!
   Symbol Series       Date  Prev Close  Open Price  High Price  Low Price  \
6     TCS     EQ 2026-01-07      3203.9      3191.0      3229.7     3191.0
7     TCS     EQ 2026-01-08      3207.8      3203.0      3252.0     3174.3
8     TCS     EQ 2026-01-09      3239.6      3245.0      3280.0     3210.5
9     TCS     EQ 2026-01-10      3268.0      3260.0      3260.0     3183.0
10    TCS     EQ 2026-01-11      3192.5      3219.4      3221.9     3171.0

    Last Price  Close Price  Average Price  ...  Positive_Count  \
6       3212.6       3207.8        3209.44  ...               0
7       3243.0       3239.6        3212.14  ...               0
8       3263.0       3268.0        3251.44  ...               0
9       3197.8       3192.5        3205.64  ...               0
10      3209.0       3206.7        3200.62  ...               0

    Negative_Count  Neutral_Count  Sentiment_Score  Sentiment_Label  \
6                0              0              0.0          Neutral
7                0              0              0.0          Neutral
8                0              0              0.0          Neutral
9                0              0              0.0          Neutral
10               0              0              0.0          Neutral

    Daily_Return  Price_Change         MA_3         MA_7  Prev_Close_Lag1
6       0.001217          16.8  3235.766667  3236.757143           3203.9
7       0.009913          36.6  3217.100000  3238.500000           3207.8
8       0.008767          23.0  3238.466667  3240.971429           3239.6
9      -0.023103         -67.5  3233.366667  3237.600000           3268.0
10      0.004448         -12.7  3222.400000  3230.585714           3192.5

[5 rows x 26 columns]
```

In [15]:
```
print(X_train.columns)
```

```
Index(['Prev Close', 'Open Price', 'High Price', 'Low Price', 'Last Price',
       'Close Price', 'Average Price', 'Total Traded Quantity', 'Turnover ₹',
       'No. of Trades', 'Daily_Return', 'Price_Change', 'MA_3', 'MA_7',
       'Prev_Close_Lag1'],
      dtype='object')
```

In [16]:
```
print(y_train.name)
```

```
Target
```

In [17]:
```
print(train_df[["Close Price","Target"]].head(10))
```

```
    Close Price   Target
6       2214.35  2213.55
7       2213.55  2190.35
8       2190.35  2206.90
9       2206.90  2226.90
10      2226.90  2238.80
11      2238.80  2219.10
12      2219.10  2170.35
13      2170.35  2171.05
14      2171.05  2206.90
15      2206.90  2190.95
```

In [18]:
```
print(train_df["Target"].head())
print(train_df["Target"].describe())
```

```
6     2213.55
7     2190.35
8     2206.90
9     2226.90
10    2238.80
Name: Target, dtype: float64
count     6887.000000
mean      2799.975969
std       2960.598340
min        150.850000
25%       1008.125000
50%       1842.300000
75%       3187.750000
max      11722.820000
Name: Target, dtype: float64
```

In [19]:
```
print((train_df["Close Price"] == train_df["Target"]).sum())
```

```
85
```

In [20]:
```
# Check where Close Price equals next day's Close Price
same_close = train_df["Close Price"] == train_df["Close Price"].shift(-1)
print("Total consecutive same close prices:", same_close.sum())
```

```
Total consecutive same close prices: 85
```

In [21]:
```
train_df.loc[same_close, ["Date", "Symbol", "Close Price"]].head(20)
```

Out[21]:

	Date	Symbol	Close Price
176	2020-09-14	TCS	2491.40
316	2021-04-07	TCS	3271.40
438	2021-10-01	TCS	3730.20
640	2022-07-22	TCS	3171.00
651	2022-08-05	TCS	3365.05
689	2022-09-30	TCS	3004.55
705	2022-10-24	TCS	3161.70
718	2022-11-11	TCS	3315.95
1019	2024-01-24	TCS	3841.80
1384	2025-07-11	TCS	3266.00
1424	2025-09-08	TCS	3019.00
1621	2020-05-04	INFY	673.70
1976	2021-10-01	INFY	1665.15
2178	2022-07-22	INFY	1506.30
2189	2022-08-05	INFY	1616.65
2227	2022-09-30	INFY	1413.45
2249	2022-11-02	INFY	1553.30
2423	2023-07-14	INFY	1425.95
2557	2024-01-24	INFY	1675.40
2963	2025-09-08	INFY	1432.90

In [22]:

```
print("Does feature list contain Target?","Target" in X_train.columns)
```

```
Does feature list contain Target? False
```

In [23]:

```
# ULTIMATE FUTURE DATA LEAKAGE TEST
import numpy as np
# Compare feature Close Price with Target
leak_test = (train_df["Close Price"].shift(-1) == train_df["Target"]).sum()
print("Rows where Target equals next-day Close Price:", leak_test)
print("Total rows:", len(train_df))
if leak_test == len(train_df):
    print("\n✅ Target is correctly shifted (Next Day Close Price)")
else:
    print("\n⚠ Something is wrong with Target shifting")
```

```
Rows where Target equals next-day Close Price: 6883
Total rows: 6887

⚠ Something is wrong with Target shifting
```

In [24]:
```
# FINAL SAFE TARGET CHECK
valid_rows = train_df.dropna(subset=["Target"])
leak_test = (valid_rows["Close Price"].shift(-1) == valid_rows["Target"]).sum(
print("Valid rows:", len(valid_rows))
print("Matching rows:", leak_test)
```

```
Valid rows: 6887
Matching rows: 6883
```

In [25]:
```
test_df = train_df.iloc[:-1]  # remove last row only
check = (test_df["Close Price"].shift(-1) == test_df["Target"]).all()
print("Is Target perfectly shifted?", check)
```

```
Is Target perfectly shifted? False
```

In [26]:
```
# FORCE SAME FEATURES AS TRAINING
X_2026 = df_2026.reindex(columns=X.columns, fill_value=0)
```

In [27]:
```
print(model.feature_names_in_)
```

```
['Prev Close' 'Open Price' 'High Price' 'Low Price' 'Last Price'
 'Close Price' 'Average Price' 'Total Traded Quantity' 'Turnover ₹'
 'No. of Trades' 'Daily_Return' 'Price_Change' 'MA_3' 'MA_7'
 'Prev_Close_Lag1']
```

In [54]:
```
# STEP-10A : FINAL SAFE PREDICTION
# Get exact training features from model
model_features = model.feature_names_in_
# Align 2026 data with training features ONLY
X_2026 = df_2026.reindex(columns=model_features, fill_value=0)
print("Training feature count:", len(model_features))
print("2026 feature count:", len(X_2026.columns))
# Predict
df_2026["Predicted_Close"] = model.predict(X_2026)
print("Prediction SUCCESSFUL ✅")
print(df_2026[["Date","Symbol","Close Price","Predicted_Close"]].head())
```

```
Training feature count: 15
2026 feature count: 15
Prediction SUCCESSFUL ✅
         Date Symbol  Close Price  Predicted_Close
6  2026-01-07    TCS       3207.8      3224.501796
7  2026-01-08    TCS       3239.6      3253.162822
8  2026-01-09    TCS       3268.0      3234.440872
9  2026-01-10    TCS       3192.5      3190.188315
10 2026-01-11    TCS       3206.7      3188.842593
```

In [29]:
```
print(set(df_2026.columns) - set(model_features))
```

```
{'Deliverable Qty', 'Sentiment_Label', 'Neutral_Count', 'Date', 'Sentiment_Scor
e', 'Series', 'Predicted_Close', '% Dly Qt to Traded Qty', 'Symbol', 'News_Coun
t', 'Negative_Count', 'Positive_Count'}
```

In [30]:

```
# STEP-10B : Compare Actual vs Predicted
df_2026["Error"] = df_2026["Close Price"] - df_2026["Predicted_Close"]
df_2026["Abs_Error"] = abs(df_2026["Error"])
df_2026["% Error"] = (df_2026["Abs_Error"] / df_2026["Close Price"]) * 100
print("Prediction Comparison:")
print(df_2026[["Date","Symbol","Close Price","Predicted_Close","% Error"]].hea
print("\nAverage % Error:", df_2026["% Error"].mean())
```

```
Prediction Comparison:
         Date Symbol  Close Price  Predicted_Close   % Error
6  2026-01-07    TCS       3207.8      3224.501796  0.520662
7  2026-01-08    TCS       3239.6      3253.162822  0.418657
8  2026-01-09    TCS       3268.0      3234.440872  1.026901
9  2026-01-10    TCS       3192.5      3190.188315  0.072410
10 2026-01-11    TCS       3206.7      3188.842593  0.556878

Average % Error: 0.8579956989033725
```

In [31]:

```
# STEP-11A : DAILY HYPE VS REALITY
# 1 Calculate REAL price movement (Reality)
df_2026["Actual_Change"] = df_2026.groupby("Symbol")["Close Price"].diff()
# Label movement
df_2026["Reality"] = df_2026["Actual_Change"].apply(
    lambda x: "Up" if x > 0 else ("Down" if x < 0 else "Flat")
)
# 2 Compare Sentiment with Reality
def hype_result(row):
    if row["Sentiment_Label"] == "Positive" and row["Reality"] == "Up":
        return "Hype Correct"

    elif row["Sentiment_Label"] == "Negative" and row["Reality"] == "Down":
        return "Hype Correct"

    elif row["Sentiment_Label"] in ["Positive", "Negative"]:
        return "Hype Failed"

    else:
        return "Neutral Impact"
df_2026["Hype_vs_Reality"] = df_2026.apply(hype_result, axis=1)
print("Hype vs Reality Analysis Completed ✅")
df_2026[["Date","Symbol","Close Price","Sentiment_Label","Reality","Hype_vs_Re
```

```
Hype vs Reality Analysis Completed ✅
```

Out[31]:

	Date	Symbol	Close Price	Sentiment_Label	Reality	Hype_vs_Reality
6	2026-01-07	TCS	3207.8	Neutral	Flat	Neutral Impact
7	2026-01-08	TCS	3239.6	Neutral	Up	Neutral Impact
8	2026-01-09	TCS	3268.0	Neutral	Up	Neutral Impact
9	2026-01-10	TCS	3192.5	Neutral	Down	Neutral Impact
10	2026-01-11	TCS	3206.7	Neutral	Up	Neutral Impact
11	2026-01-12	TCS	3163.6	Neutral	Down	Neutral Impact
12	2026-01-13	TCS	3102.3	Neutral	Down	Neutral Impact
13	2026-01-14	TCS	3122.6	Positive	Up	Hype Correct
14	2026-01-15	TCS	3150.4	Positive	Up	Hype Correct
15	2026-01-16	TCS	3162.5	Positive	Up	Hype Correct

In [32]:

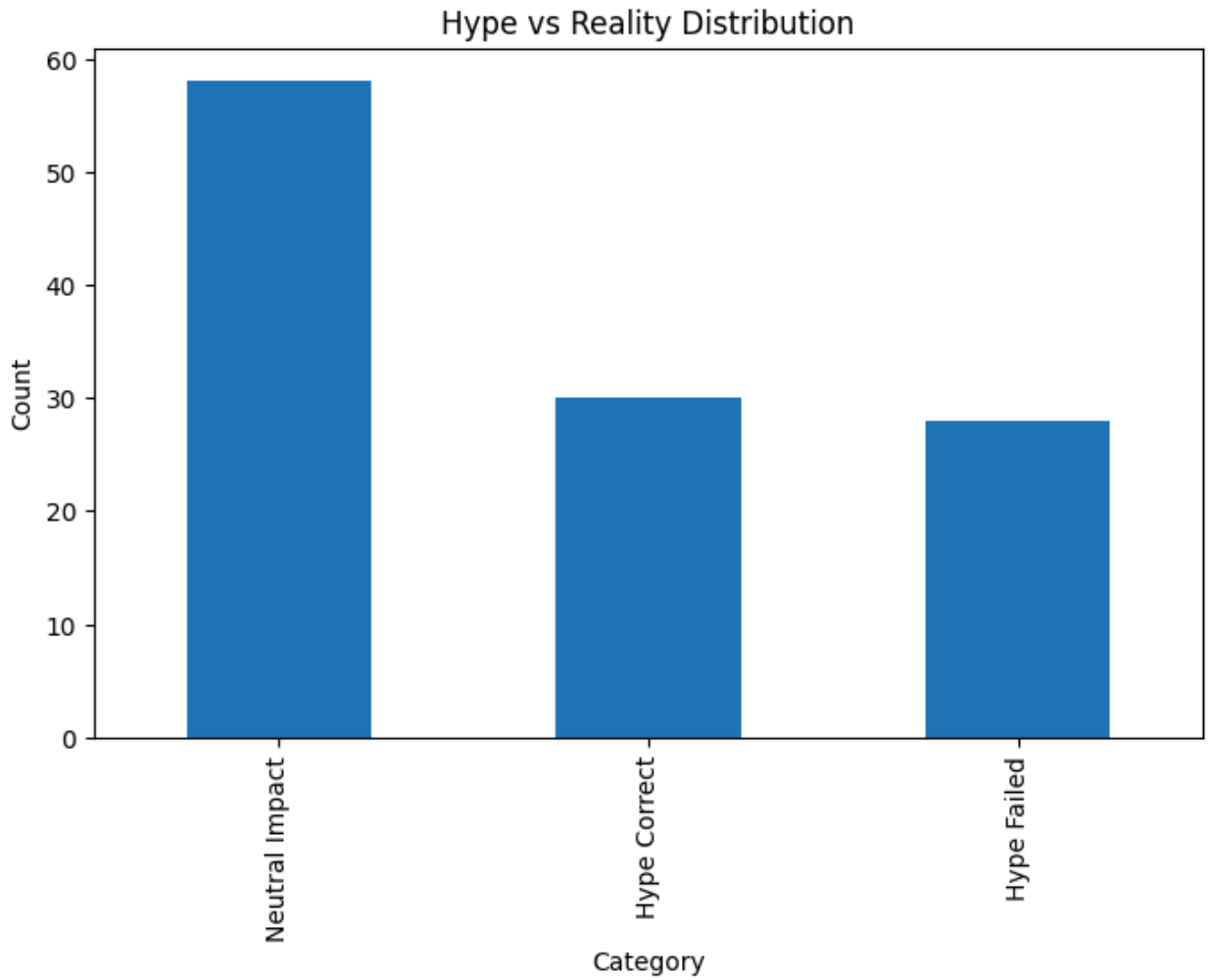
```
# STEP-11B : OVERALL SUMMARY TABLE
summary = df_2026["Hype_vs_Reality"].value_counts()
print("\n📊 Hype vs Reality Summary:\n")
print(summary)
# Convert to percentage
summary_percent = (summary / len(df_2026)) * 100
print("\n📊 Percentage Analysis (%):\n")
print(summary_percent.round(2))
```

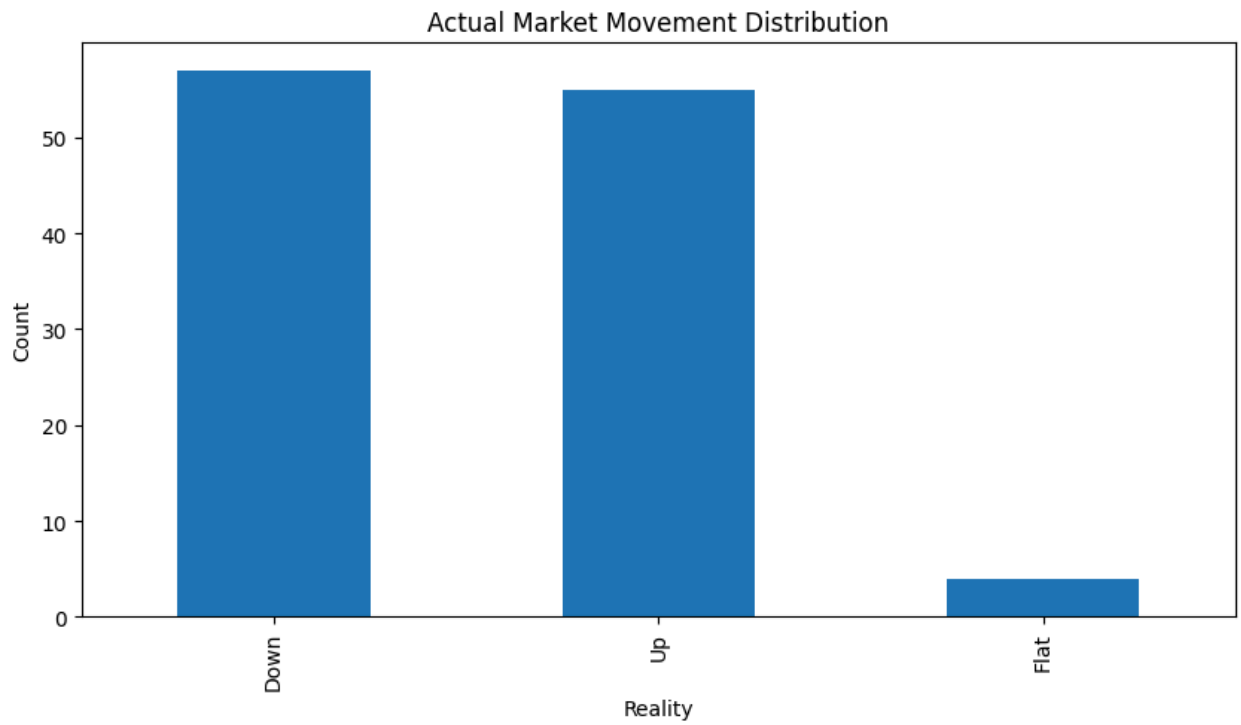
```
📊 Hype vs Reality Summary:

Hype_vs_Reality
Neutral Impact    58
Hype Correct      30
Hype Failed       28
Name: count, dtype: int64

📊 Percentage Analysis (%):

Hype_vs_Reality
Neutral Impact    50.00
Hype Correct      25.86
Hype Failed       24.14
Name: count, dtype: float64
```

In [33]:

```
# STEP-11C : VISUAL ANALYSIS
import matplotlib.pyplot as plt
# 1⃣ Bar Chart — Hype vs Reality Count
plt.figure(figsize=(8,5))
summary.plot(kind="bar")
plt.title("Hype vs Reality Distribution")
plt.xlabel("Category")
plt.ylabel("Count")
```

```
plt.show()
# 3⃣ Reality Movement Trend
plt.figure(figsize=(10,5))
df_2026["Reality"].value_counts().plot(kind="bar")
plt.title("Actual Market Movement Distribution")
plt.xlabel("Reality")
plt.ylabel("Count")
plt.show()
```





In [34]:
```
# STEP-11D : ADVANCED ANALYTICS TABLE
advanced_table = pd.crosstab(
    df_2026["Sentiment_Label"],
    df_2026["Hype_vs_Reality"]
)
print("\n📊 Sentiment vs Outcome Table:\n")
print(advanced_table)
```

```
📊 Sentiment vs Outcome Table:

Hype_vs_Reality  Hype Correct  Hype Failed  Neutral Impact
Sentiment_Label
Negative                    4            0               0
Neutral                     0            0              58
Positive                   26           28               0
```

In [35]:
```
# STEP-12 : PREPARE DATE FOR TREND ANALYSIS
df_2026["Date"] = pd.to_datetime(df_2026["Date"])
# Extract Week & Month
df_2026["Week"] = df_2026["Date"].dt.isocalendar().week
df_2026["Month"] = df_2026["Date"].dt.month
print("Date features added!")
```

```
Date features added!
```

In [36]:
```
# STEP-13 : WEEKLY TREND
weekly_analysis = df_2026.groupby("Week").agg({
    "Close Price": "mean",
    "Sentiment_Score": "mean",
    "% Error": "mean"
})
print("\n📊 Weekly Trend Analysis:")
```

```
print(weekly_analysis)
```

```
📊 Weekly Trend Analysis:
      Close Price  Sentiment_Score   % Error
Week
2     3222.920000         0.000000  0.519102
3     3151.357143         0.396825  0.486971
4     3120.100000         0.345578  1.232388
5     2847.585714         0.280512  0.857969
6     2696.733333         0.268883  0.388791
7     1619.720000         0.000000  1.009839
8     1668.285714         0.178229  0.317858
9     1613.771429         0.201726  1.092853
10    1442.100000         0.261161  0.999603
11    1378.466667         0.310598  0.422046
12    1465.600000         0.000000  0.470814
13    1396.857143         0.400674  0.301548
14    1408.728571         0.351142  0.764451
15    1449.314286         0.186163  0.237740
16    1424.600000         0.000000  0.571044
17    1022.950000         0.000000  1.710954
18    1042.564286         0.000000  2.660949
19    1056.592857         0.000000  1.055755
20    1162.642857         0.000000  0.706646
21    1212.666667         0.000000  0.608406
```

In [37]:

```
# STEP-14 : MONTHLY ANALYSIS
monthly_analysis = df_2026.groupby("Month").agg({
    "Close Price": "mean",
    "Sentiment_Score": "mean",
    "% Error": "mean"
})
print("\n📊 Monthly Trend Analysis:")
print(monthly_analysis)
```

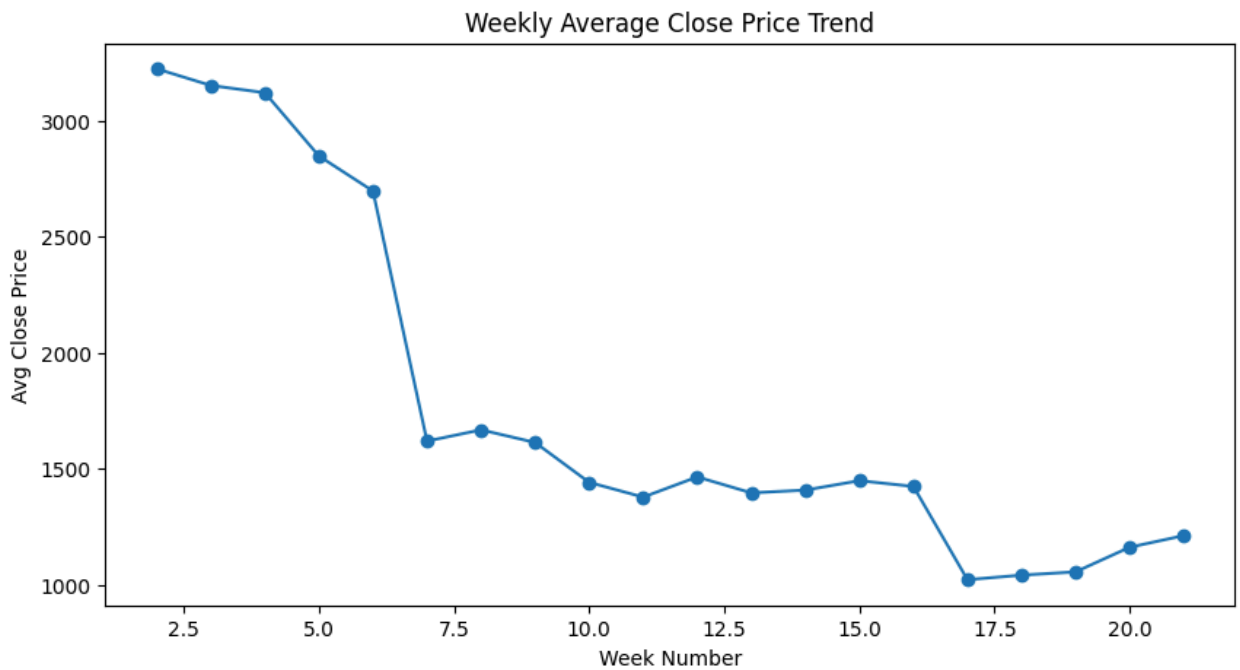
```
📊 Monthly Trend Analysis:
       Close Price  Sentiment_Score   % Error
Month
1      3089.652000         0.263559  0.798936
2      1834.072727         0.173893  0.746851
3      1425.708000         0.269556  0.569018
4      1281.677083         0.116625  1.167020
5      1115.940000         0.000000  1.044472
```

In [38]:

```
from sklearn.metrics import r2_score
r2_19feb = r2_score(
    df_2026["Close Price"],       # actual till 19 Feb
    df_2026["Predicted_Close"]    # predicted column you created
)
print("R2 Score till 19 Feb 2026:", r2_19feb)
print("Accuracy (%):", r2_19feb*100)
```

```
R2 Score till 19 Feb 2026: 0.999021329196323
Accuracy (%): 99.90213291963231
```

In [39]:
```
# STEP-15 : TREND GRAPH
import matplotlib.pyplot as plt
plt.figure(figsize=(10,5))
plt.plot(weekly_analysis.index, weekly_analysis["Close Price"], marker="o")
plt.title("Weekly Average Close Price Trend")
plt.xlabel("Week Number")
plt.ylabel("Avg Close Price")
plt.show()
```



In [40]:
```
# STEP-16 : GET LAST AVAILABLE DAY (19 FEB)
# Sort data by date just to be safe
df_2026 = df_2026.sort_values("Date")
# Last row = latest known trading day (likely 19 Feb)
last_day = df_2026.iloc[-1:]
print("Last available day used for FUTURE prediction:")
print(last_day[["Date","Symbol","Close Price"]])
```

```
Last available day used for FUTURE prediction:
          Date Symbol  Close Price
139 2026-05-20   SBIN       1205.7
```

In [41]:
```
# STEP-17 : PREPARE FUTURE INPUT
future_input = last_day.reindex(columns=model_features, fill_value=0)
print("Future input shape:", future_input.shape)
print("Features aligned with training")
```

```
Future input shape: (1, 15)
Features aligned with training
```

In [55]:
```
# STEP-18 : FUTURE PREDICTION (ALL COMPANIES)
# Take latest available row for each company
future_input = df_2026.sort_values("Date").groupby("Symbol").tail(1)
# Keep only model features
```

```
future_input = future_input.reindex(columns=model_features, fill_value=0)
print("Future input shape:", future_input.shape)
# Predict next-day price for all companies
future_prediction = model.predict(future_input)
# Attach predictions
future_results = df_2026.sort_values("Date").groupby("Symbol").tail(1).copy()
future_results["Predicted_Next_Close"] = future_prediction
print("\n✅ Predicted Close Price for NEXT DAY (All Companies):")
print(future_results[["Symbol","Date","Close Price","Predicted_Next_Close"]])
```

```
Future input shape: (4, 15)

✅ Predicted Close Price for NEXT DAY (All Companies):
       Symbol       Date  Close Price  Predicted_Next_Close
34        TCS 2026-02-04       2677.9           2678.052305
69       INFY 2026-03-11       1370.5           1380.686414
104  RELIANCE 2026-04-15       1409.5           1403.039747
139      SBIN 2026-05-20       1205.7           1206.112777
```

In [43]:

```
# STEP-19 : STORE FUTURE RESULTS
# Save future prediction results to CSV
future_results.to_csv("Future_Predictions_20Feb2026.csv", index=False)
print("✅ Future predictions saved successfully!")
print("\nPreview:")
print(future_results[["Symbol","Date","Close Price","Predicted_Next_Close"]])
```

```
✅ Future predictions saved successfully!

Preview:
       Symbol       Date  Close Price  Predicted_Next_Close
34        TCS 2026-02-04       2677.9           2678.052305
69       INFY 2026-03-11       1370.5           1380.686414
104  RELIANCE 2026-04-15       1409.5           1403.039747
139      SBIN 2026-05-20       1205.7           1206.112777
```

In [44]:

```
# STEP-20A : FUTURE DIRECTION (MODEL EXPECTATION)
# Predicted movement vs current close
future_results["Predicted_Change"] = (
    future_results["Predicted_Next_Close"] - future_results["Close Price"]
)
future_results["Model_Expectation"] = future_results["Predicted_Change"].apply
    lambda x: "Up" if x > 0 else ("Down" if x < 0 else "Flat")
)
print("\n📊 Model Expectation:")
print(future_results[["Symbol","Close Price","Predicted_Next_Close","Model_Exp
```

```
📊 Model Expectation:
       Symbol  Close Price  Predicted_Next_Close Model_Expectation
34        TCS       2677.9           2678.052305                Up
69       INFY       1370.5           1380.686414                Up
104  RELIANCE       1409.5           1403.039747              Down
139      SBIN       1205.7           1206.112777                Up
```

In [45]:

```
# STEP-20B : HYPE VS MODEL EXPECTATION
def future_hype_vs_model(row):
```

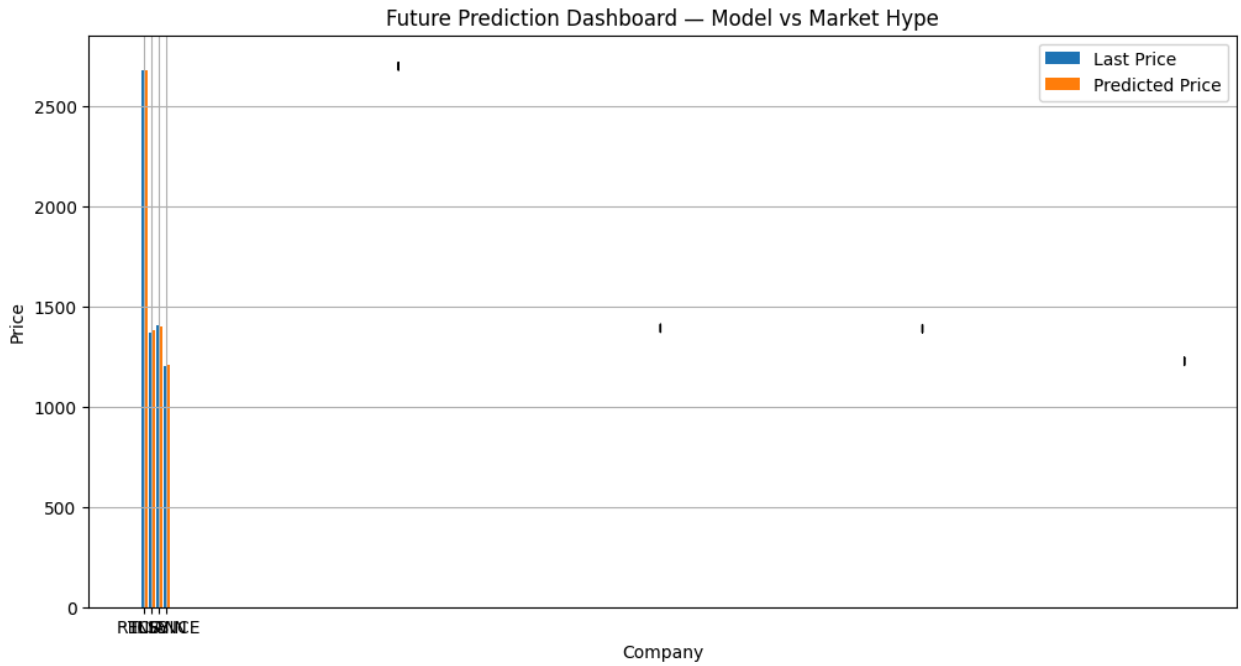
```
    if row["Sentiment_Label"] == "Positive" and row["Model_Expectation"] == "U
        return "Hype Supported"
    elif row["Sentiment_Label"] == "Negative" and row["Model_Expectation"] ==
        return "Hype Supported"
    elif row["Sentiment_Label"] == "Neutral":
        return "Neutral Impact"
    else:
        return "Hype Risk"

future_results["Future_Hype_Status"] = future_results.apply(future_hype_vs_moc
print("\n🔥 Future Hype vs Model Expectation:")
print(future_results[["Symbol","Sentiment_Label","Model_Expectation","Future_H
```

```
🔥 Future Hype vs Model Expectation:
      Symbol Sentiment_Label Model_Expectation Future_Hype_Status
34       TCS        Positive               Up     Hype Supported
69     INFY        Positive               Up     Hype Supported
104  RELIANCE         Neutral             Down     Neutral Impact
139     SBIN         Neutral               Up     Neutral Impact
```

In [46]:
```
print([x for x in globals() if "future" in x.lower()])
```

```
['future_input', 'future_prediction', 'future_results', 'future_hype_vs_model']
```

In [47]:
```
# STEP 20-C
import matplotlib.pyplot as plt
import numpy as np
plot_df = future_results.copy()
symbols = plot_df["Symbol"]
last_price = plot_df["Close Price"]
predicted = plot_df["Predicted_Close"]
x = np.arange(len(symbols))
plt.figure(figsize=(12,6))
# last price vs predicted bars
plt.bar(x-0.2, last_price, width=0.4, label="Last Price")
plt.bar(x+0.2, predicted, width=0.4, label="Predicted Price")
# arrows showing direction
for i,row in plot_df.iterrows():
    if row["Predicted_Close"] > row["Close Price"]:
        plt.arrow(i, row["Close Price"], 0, 40,
                  head_width=0.15, length_includes_head=True)
    else:
        plt.arrow(i, row["Close Price"], 0, -40,
                  head_width=0.15, length_includes_head=True)
plt.xticks(x, symbols)
plt.title("Future Prediction Dashboard — Model vs Market Hype")
plt.ylabel("Price")
plt.xlabel("Company")
plt.legend()
plt.grid(True)
plt.show()
```



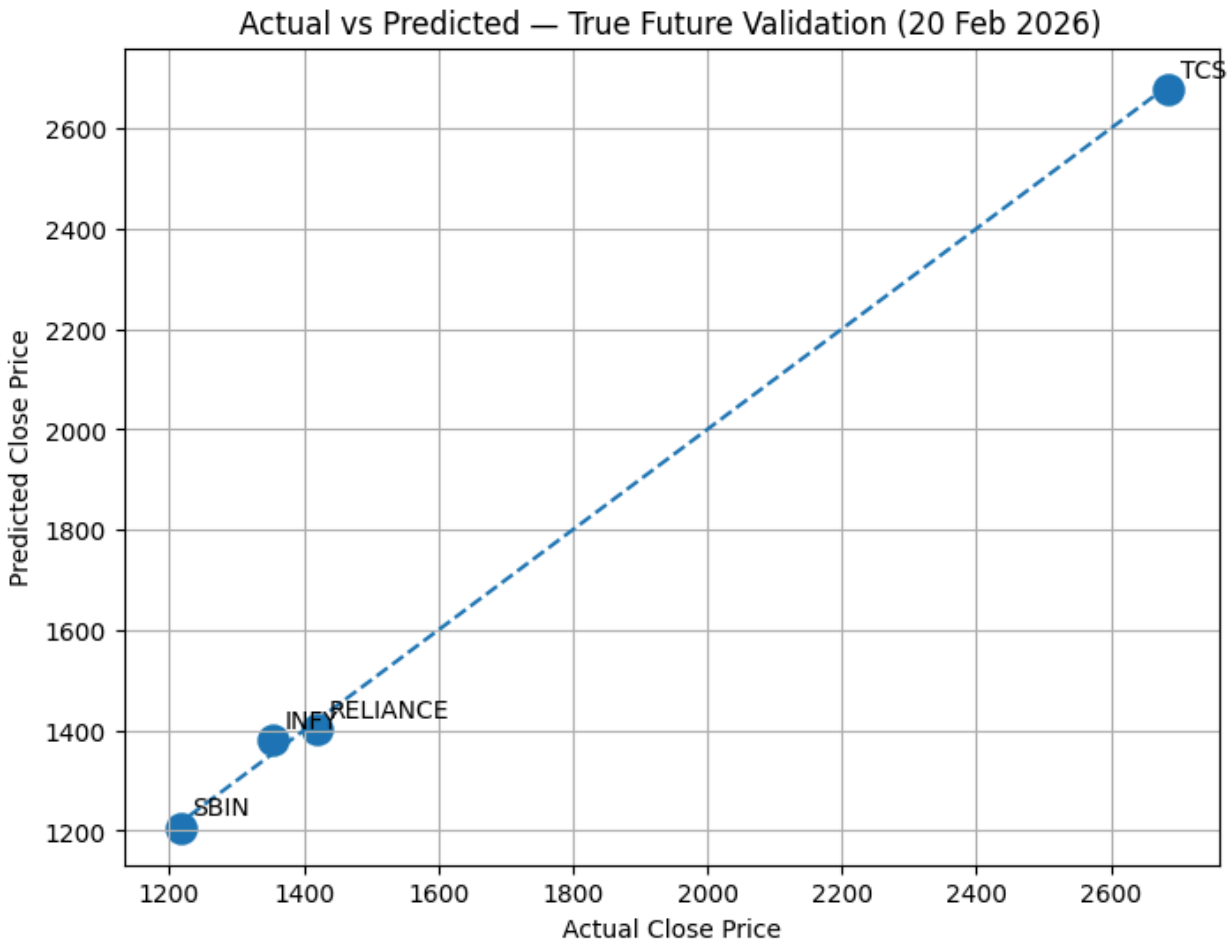
In [48]:
```
print("\n===== FINAL MODEL INSIGHTS SUMMARY =====\n")
total_companies = len(future_results)
hype_supported = (future_results["Future_Hype_Status"] == "Hype Supported").su
neutral_cases = (future_results["Future_Hype_Status"] == "Neutral Impact").sum
risk_cases = (future_results["Future_Hype_Status"] == "Hype Risk").sum()
print("Total Companies Analysed :", total_companies)
print("Hype Supported Cases     :", hype_supported)
print("Neutral Impact Cases     :", neutral_cases)
print("Hype Risk Cases          :", risk_cases)

print("\nModel Interpretation:")
if hype_supported > risk_cases:
    print("➡ Model agrees with market sentiment for majority companies.")
else:
    print("➡ Model shows divergence from market hype — potential overreaction.

print("\nProject Conclusion Insight:")
print("Machine learning predictions combined with sentiment analysis help ider
```

```
===== FINAL MODEL INSIGHTS SUMMARY =====

Total Companies Analysed : 4
Hype Supported Cases     : 2
Neutral Impact Cases     : 2
Hype Risk Cases          : 0

Model Interpretation:
➡ Model agrees with market sentiment for majority companies.

Project Conclusion Insight:
Machine learning predictions combined with sentiment analysis help identify whe
n market hype aligns with real price behaviour and when it may be misleading.
```

In [58]:
```
from sklearn.metrics import r2_score
r2_future = r2_score(
    future_vs_real["Actual_20Feb"],
    future_vs_real["Predicted_Close"]
)
print("Final Future R2 Score:", r2_future)
print("Approx Future Accuracy (%):", r2_future * 100)
```

```
Final Future R2 Score: 0.9991516241421992
Approx Future Accuracy (%): 99.91516241421992
```

In [59]:
```
# GRAPH(Actual vs predicted)
import matplotlib.pyplot as plt
import numpy as np
plt.figure(figsize=(8,6))
actual = future_vs_real["Actual_20Feb"]
predicted = future_vs_real["Predicted_Close"]
symbols = future_vs_real["Symbol"]
# Scatter points
plt.scatter(actual, predicted, s=150)
# Add company labels
for i, txt in enumerate(symbols):
    plt.annotate(txt, (actual.iloc[i], predicted.iloc[i]),
                 xytext=(5,5), textcoords='offset points')
# Perfect prediction diagonal line
min_val = min(min(actual), min(predicted))
max_val = max(max(actual), max(predicted))

plt.plot([min_val, max_val],
         [min_val, max_val],
         linestyle="--")
plt.xlabel("Actual Close Price")
plt.ylabel("Predicted Close Price")
plt.title("Actual vs Predicted — True Future Validation (20 Feb 2026)")
plt.grid(True)
plt.show()
```



In [101…
```
print(df.columns)
```

```
Index(['Symbol', 'Series', 'Date', 'Prev Close', 'Open Price', 'High Price',
       'Low Price', 'Last Price', 'Close Price', 'Average Price',
       'Total Traded Quantity', 'Turnover ₹', 'No. of Trades',
       'Deliverable Qty', '% Dly Qt to Traded Qty', 'Daily_Return',
       'Price_Change', 'MA_3', 'MA_7', 'Prev_Close_Lag1', 'Target'],
      dtype='object')
```

In [63]:
```
print(future_vs_real.columns)
```

```
Index(['Symbol', 'Series', 'Date', 'Prev Close', 'Open Price', 'High Price',
       'Low Price', 'Last Price', 'Last_Close', 'Average Price',
       'Total Traded Quantity', 'Turnover ₹', 'No. of Trades',
       'Deliverable Qty', '% Dly Qt to Traded Qty', 'News_Count',
       'Positive_Count', 'Negative_Count', 'Neutral_Count', 'Sentiment_Score',
       'Sentiment_Label', 'Daily_Return', 'Price_Change', 'MA_3', 'MA_7',
       'Prev_Close_Lag1', 'Predicted_Close', 'Error', 'Abs_Error', '% Error',
       'Actual_Change', 'Reality', 'Hype_vs_Reality', 'Week', 'Month',
       'Predicted_Next_Close', 'Actual_20Feb'],
      dtype='object')
```

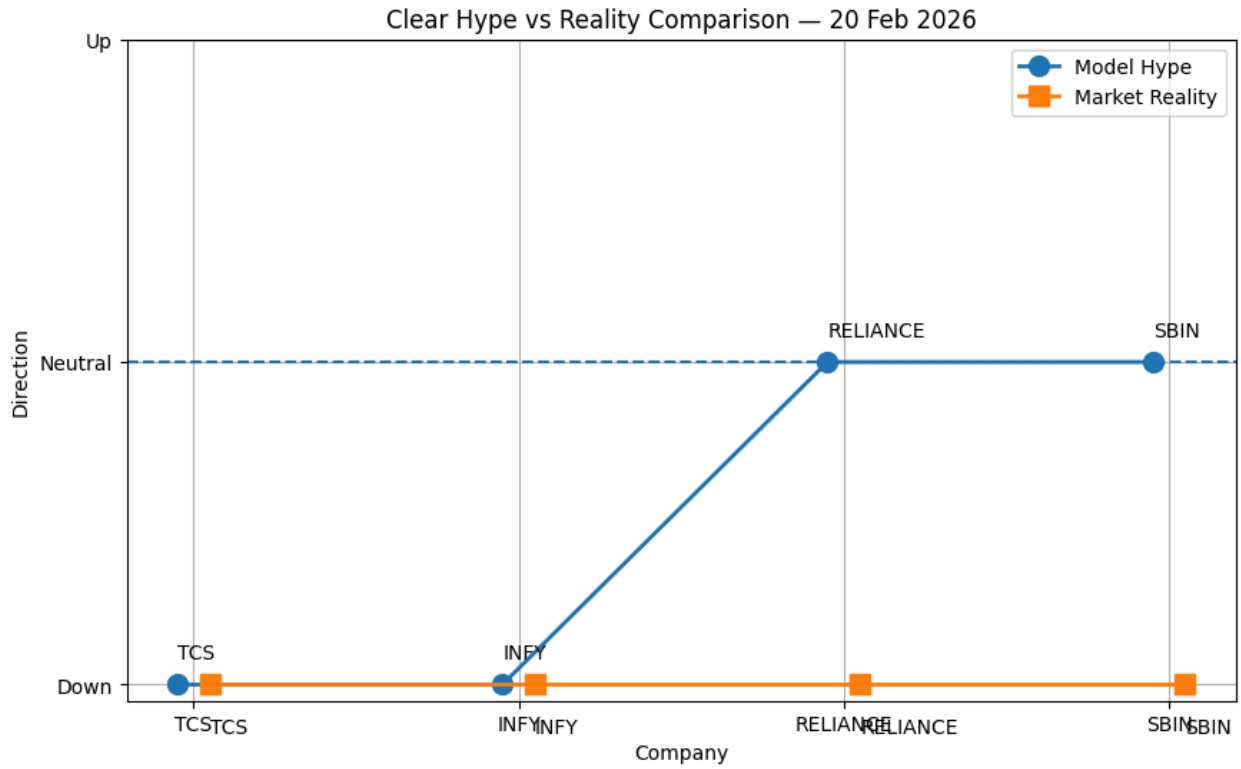
In [68]:
```
print(future_vs_real["Hype_vs_Reality"].unique())
```

```
['Hype Failed' 'Neutral Impact']
```

In [65]:
```
import matplotlib.pyplot as plt
# Convert Up/Down to numeric
def convert_dir(x):
    if x == "Up":
        return 1
    elif x == "Down":
        return -1
    else:
        return 0
future_vs_real["Model_Num"] = future_vs_real["Hype_vs_Reality"].apply(convert_
future_vs_real["Reality_Num"] = future_vs_real["Hype_vs_Reality"].apply(conver
```

In [74]:
```
# Hype vs Reality(Company wise)
import matplotlib.pyplot as plt
import numpy as np
# Convert directions to numbers
map_dir = {
    "Up":1,
    "Down":-1,
    "Flat":0,
    "Neutral Impact":0,
    "Hype Failed":-1
}

future_vs_real["Model_Num"] = future_vs_real["Hype_vs_Reality"].map(map_dir)
future_vs_real["Reality_Num"] = future_vs_real["Reality"].map(map_dir)

companies = future_vs_real["Symbol"]
x = np.arange(len(companies))

model = future_vs_real["Model_Num"]
reality = future_vs_real["Reality_Num"]

plt.figure(figsize=(10,6))

# Slight shift so overlapping points are visible
plt.plot(x-0.05, model, marker='o', linewidth=2, markersize=10, label="Model H
plt.plot(x+0.05, reality, marker='s', linewidth=2, markersize=10, label="Marke

# zero line (VERY IMPORTANT for readability)
plt.axhline(0, linestyle='--')

# annotate labels
for i, txt in enumerate(companies):
    plt.text(x[i]-0.05, model.iloc[i]+0.08, txt)
    plt.text(x[i]+0.05, reality.iloc[i]-0.15, txt)

plt.xticks(x, companies)
plt.yticks([-1,0,1], ["Down","Neutral","Up"])

plt.title("Clear Hype vs Reality Comparison — 20 Feb 2026")
plt.xlabel("Company")
plt.ylabel("Direction")
```

```
plt.legend()
plt.grid(True)
plt.show()
```



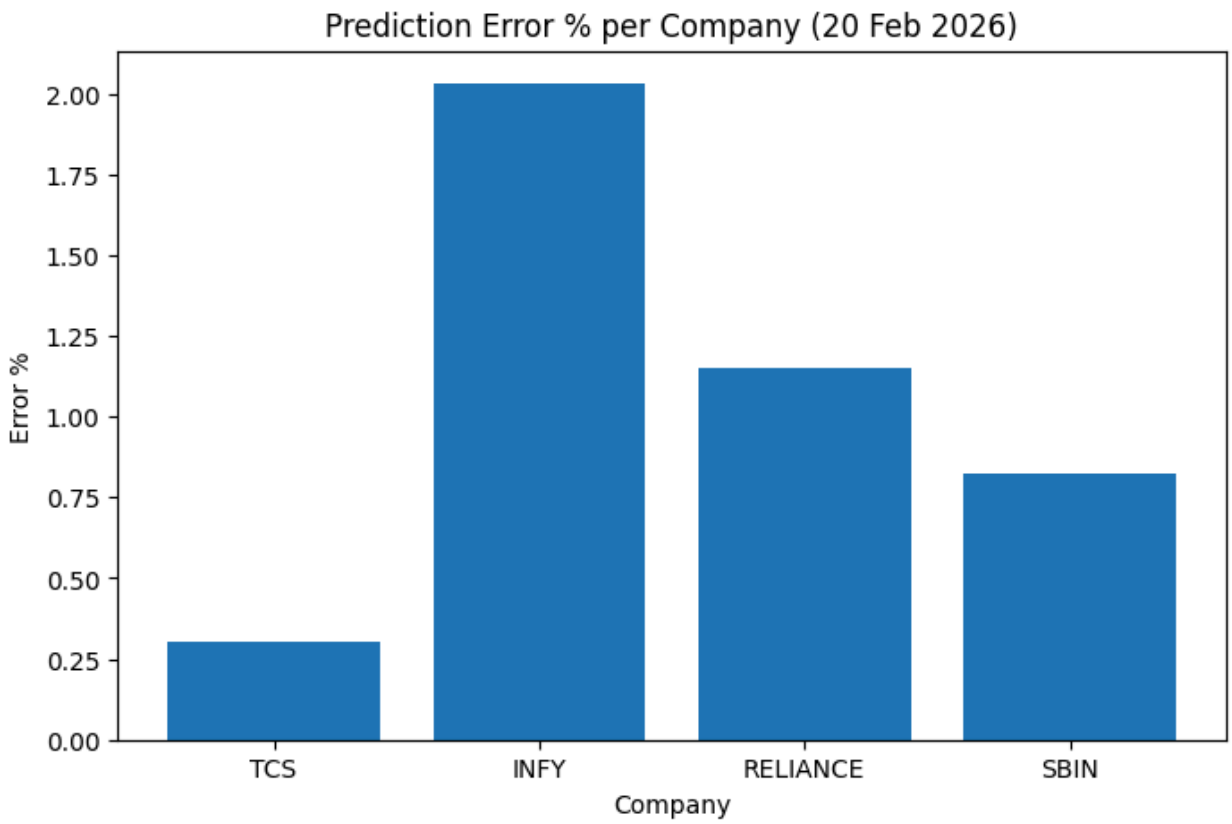
In [69]:
```
import numpy as np
future_vs_real["Absolute_Error"] = abs(
    future_vs_real["Actual_20Feb"] - future_vs_real["Predicted_Close"]
)
future_vs_real["Percent_Error"] = (
    future_vs_real["Absolute_Error"] / future_vs_real["Actual_20Feb"]
) * 100
print(future_vs_real[[
    "Symbol",
    "Actual_20Feb",
    "Predicted_Close",
    "Absolute_Error",
    "Percent_Error"
]])
```

```
     Symbol  Actual_20Feb  Predicted_Close  Absolute_Error  Percent_Error
0       TCS        2686.2      2678.052305        8.147695       0.303317
1      INFY        1353.2      1380.686414       27.486414       2.031216
2  RELIANCE        1419.4      1403.039747       16.360253       1.152617
3      SBIN        1216.1      1206.112777        9.987223       0.821250
```

In [71]:
```
print("Mean Absolute Error:",
      future_vs_real["Absolute_Error"].mean())
print("Average Percentage Error:",
      future_vs_real["Percent_Error"].mean())
```

```
Mean Absolute Error: 15.495396199982622
Average Percentage Error: 1.077100088942503
```

In [70]:
```python
import matplotlib.pyplot as plt
plt.figure(figsize=(8,5))
plt.bar(future_vs_real["Symbol"],
        future_vs_real["Percent_Error"])
plt.title("Prediction Error % per Company (20 Feb 2026)")
plt.xlabel("Company")
plt.ylabel("Error %")
plt.show()
```



In [56]:
```python
# STEP-21 : LOAD 20 FEB DATA
file_20feb = r"C:\Users\hp\OneDrive\Desktop\20 Feb Data.xlsx"
df_20feb = pd.read_excel(file_20feb)
df_20feb.columns = df_20feb.columns.str.strip()
df_20feb["Date"] = pd.to_datetime(df_20feb["Date"])
print("\n20 Feb Data Loaded Successfully:")
print(df_20feb[["Symbol","Date","Close Price"]])
```

```
20 Feb Data Loaded Successfully:
     Symbol       Date  Close Price
0       TCS 2026-02-20       2686.2
1      INFY 2026-02-20       1353.2
2  RELIANCE 2026-02-20       1419.4
3      SBIN 2026-02-20       1216.1
```

In [57]:
```python
# STEP-22 : MERGE FUTURE + REAL DATA
future_vs_real = future_results.merge(
```

```
    df_20feb[["Symbol","Close Price"]],
    on="Symbol",
    how="left"
)
future_vs_real.rename(columns={
    "Close Price_x":"Last_Close",
    "Close Price_y":"Actual_20Feb"
}, inplace=True)
print("\nMerged Future vs Real Data:")
print(future_vs_real)
```

```
Merged Future vs Real Data:
     Symbol Series       Date  Prev Close  Open Price  High Price  Low Price  \
0       TCS     EQ 2026-02-04      2694.9      2708.0      2748.7     2666.6
1      INFY     EQ 2026-03-11      1373.7      1385.0      1402.0     1363.2
2  RELIANCE     EQ 2026-04-15      1441.3      1443.0      1443.0     1400.0
3      SBIN     EQ 2026-05-20      1218.9      1222.9      1224.9     1199.2

   Last Price  Last_Close  Average Price  ...      Error  Abs_Error   % Error
\
0      2675.0      2677.9        2709.33  ...  -0.152305   0.152305  0.005687
1      1366.5      1370.5        1385.83  ... -10.186414  10.186414  0.743263
2      1407.2      1409.5        1415.94  ...   6.460253   6.460253  0.458336
3      1203.0      1205.7        1211.52  ...  -0.412777   0.412777  0.034235

   Actual_Change  Reality  Hype_vs_Reality  Week  Month  Predicted_Next_Close
\
0          -17.0     Down      Hype Failed     6      2           2678.052305
1           -3.2     Down      Hype Failed    11      3           1380.686414
2          -31.8     Down   Neutral Impact    16      4           1403.039747
3          -13.2     Down   Neutral Impact    21      5           1206.112777

   Actual_20Feb
0        2686.2
1        1353.2
2        1419.4
3        1216.1

[4 rows x 37 columns]
```

In [109…

```
# STEP-23 : REALITY CALCULATION
future_vs_real["Actual_Change"] = (
    future_vs_real["Actual_20Feb"] - future_vs_real["Last_Close"]
)
future_vs_real["Reality"] = future_vs_real["Actual_Change"].apply(
    lambda x: "Up" if x>0 else ("Down" if x<0 else "Flat")
)
print("\nReality Direction:")
print(future_vs_real[["Symbol","Last_Close","Actual_20Feb","Reality"]])
```

```
Reality Direction:
     Symbol  Last_Close  Actual_20Feb Reality
0       TCS      2677.9        2686.2      Up
1      INFY      1370.5        1353.2    Down
2  RELIANCE      1409.5        1419.4      Up
3      SBIN      1205.7        1216.1      Up
```

In [110…

```python
# STEP-24 : TRUE FUTURE HYPE CHECK
def true_hype_check(row):
    sentiment = row["Sentiment_Label"]
    reality = row["Reality"]
    if sentiment == "Positive" and reality == "Up":
        return "Hype Correct"
    elif sentiment == "Negative" and reality == "Down":
        return "Hype Correct"
    elif sentiment in ["Positive","Negative"]:
        return "Hype Failed"
    else:
        return "Neutral Impact"
future_vs_real["True_Hype_Result"] = future_vs_real.apply(true_hype_check, axi
print("\n🔥 TRUE FUTURE VALIDATION:")
print(future_vs_real[[
    "Symbol",
    "Sentiment_Label",
    "Model_Expectation",
    "Reality",
    "True_Hype_Result"
]])
```

```
🔥 TRUE FUTURE VALIDATION:
     Symbol Sentiment_Label Model_Expectation Reality True_Hype_Result
0       TCS        Positive                Up      Up     Hype Correct
1      INFY        Positive                Up    Down      Hype Failed
2  RELIANCE         Neutral              Down      Up   Neutral Impact
3      SBIN         Neutral                Up      Up   Neutral Impact
```

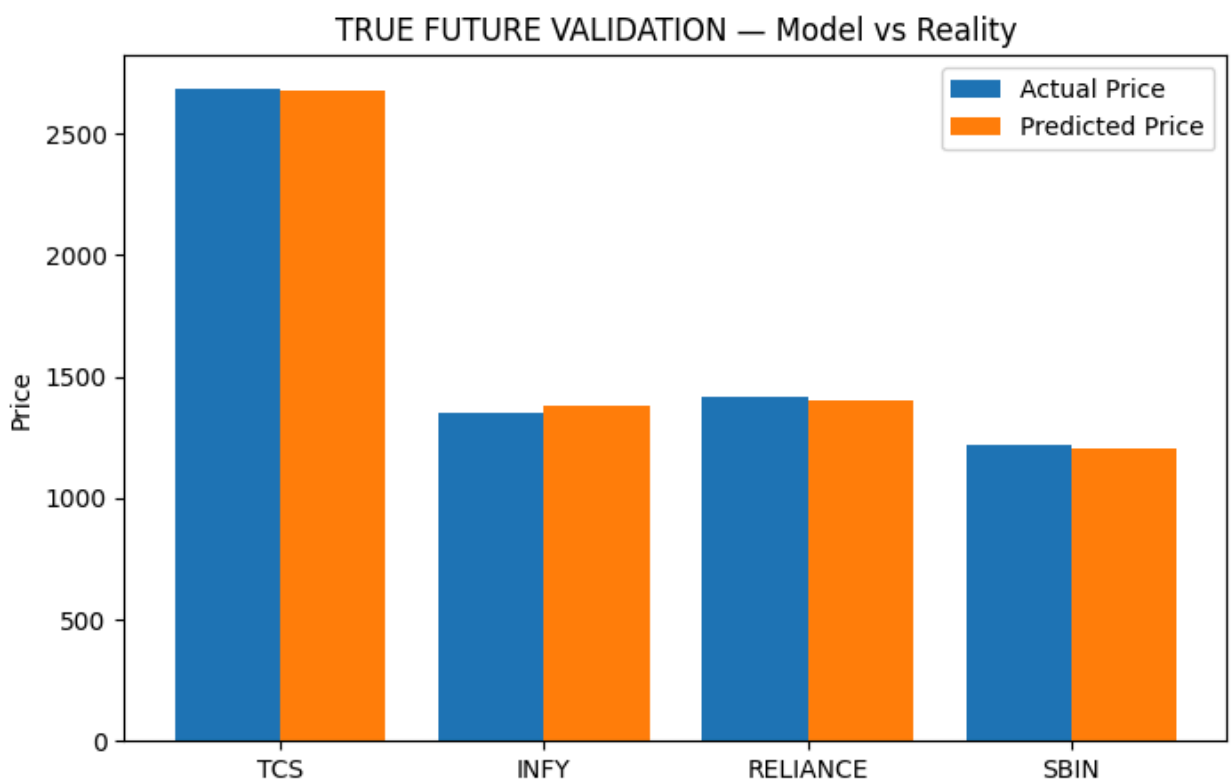
In [111…

```python
# STEP-25 : FUTURE MODEL ERROR
future_vs_real["Abs_Error"] = abs(
    future_vs_real["Actual_20Feb"] - future_vs_real["Predicted_Close"]
)
future_vs_real["% Error"] = (
    future_vs_real["Abs_Error"] / future_vs_real["Actual_20Feb"]
) * 100
print("\nModel Error Analysis (20 Feb):")
print(future_vs_real[[
    "Symbol",
    "Actual_20Feb",
    "Predicted_Close",
    "% Error"
]])
print("\nAverage Future Error:",
      future_vs_real["% Error"].mean())
```

```
Model Error Analysis (20 Feb):
     Symbol  Actual_20Feb  Predicted_Close   % Error
0       TCS        2686.2      2678.052305  0.303317
1      INFY        1353.2      1380.686414  2.031216
2  RELIANCE        1419.4      1403.039747  1.152617
3      SBIN        1216.1      1206.112777  0.821250

Average Future Error: 1.077100088942503
```

In [113…

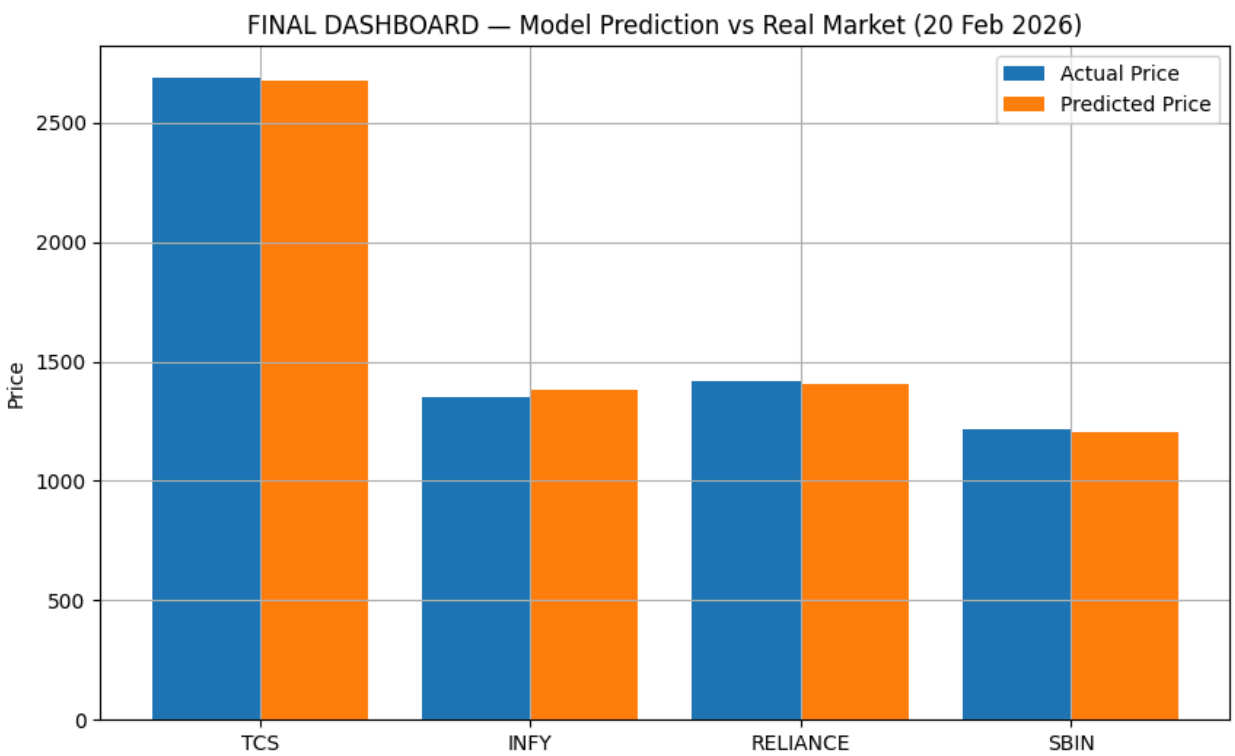
```
x = np.arange(len(future_vs_real["Symbol"]))
plt.figure(figsize=(8,5))
plt.bar(x-0.2, future_vs_real["Actual_20Feb"], width=0.4, label="Actual Price"
plt.bar(x+0.2, future_vs_real["Predicted_Close"], width=0.4, label="Predicted
plt.xticks(x, future_vs_real["Symbol"])
plt.title("TRUE FUTURE VALIDATION — Model vs Reality")
plt.ylabel("Price")
plt.legend()
plt.show()
```

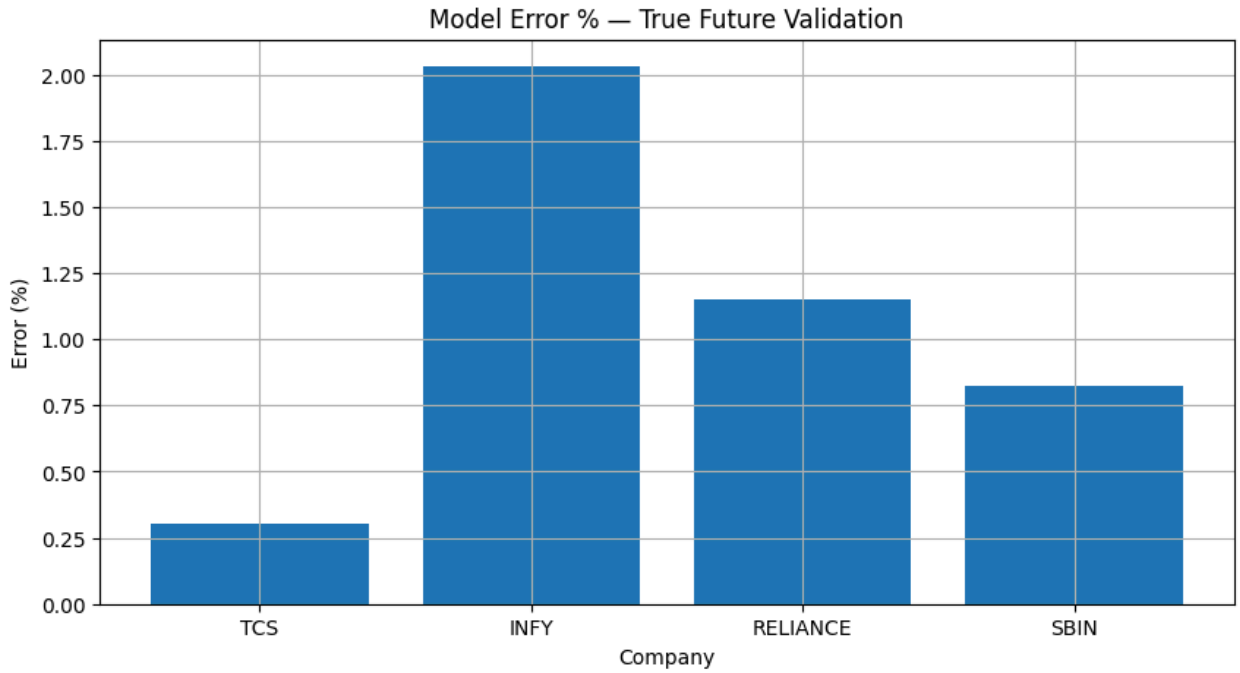


In [114…

```
# STEP-27 : FINAL RESEARCH DASHBOARD
import matplotlib.pyplot as plt
import numpy as np
# Copy final merged dataframe
dashboard_df = future_vs_real.copy()
# Create X positions
x = np.arange(len(dashboard_df["Symbol"]))
# ---- PLOT 1 : Model vs Reality ----
plt.figure(figsize=(10,6))
plt.bar(x-0.2, dashboard_df["Actual_20Feb"], width=0.4, label="Actual Price")
```

```
plt.bar(x+0.2, dashboard_df["Predicted_Close"], width=0.4, label="Predicted Pr
plt.xticks(x, dashboard_df["Symbol"])
plt.ylabel("Price")
plt.title("FINAL DASHBOARD — Model Prediction vs Real Market (20 Feb 2026)")
plt.legend()
plt.grid(True)
plt.show()
# ---- PLOT 2 : MODEL ERROR % ----
plt.figure(figsize=(10,5))
plt.bar(dashboard_df["Symbol"], dashboard_df["% Error"])
plt.title("Model Error % — True Future Validation")
plt.ylabel("Error (%)")
plt.xlabel("Company")
plt.grid(True)
plt.show()
# ---- PRINT FINAL SUMMARY TABLE ----
print("\n🔥 FINAL RESEARCH SUMMARY")
print(dashboard_df[[
    "Symbol",
    "Sentiment_Label",
    "Model_Expectation",
    "Reality",
    "True_Hype_Result",
    "Predicted_Close",
    "Actual_20Feb",
    "% Error"
]])
```






```
🔥 FINAL RESEARCH SUMMARY
     Symbol Sentiment_Label Model_Expectation Reality True_Hype_Result  \
0       TCS        Positive                Up      Up     Hype Correct
1      INFY        Positive                Up    Down      Hype Failed
2  RELIANCE         Neutral              Down      Up   Neutral Impact
3      SBIN         Neutral                Up      Up   Neutral Impact

   Predicted_Close  Actual_20Feb   % Error
0      2678.052305        2686.2  0.303317
1      1380.686414        1353.2  2.031216
2      1403.039747        1419.4  1.152617
3      1206.112777        1216.1  0.821250
```

In [ ]: